

« Ma Collection »

Backend : Python · FastAPI · SQLAlchemy **Frontend** : React 18+ · TypeScript · Vite

Public : B2 **Modalité** : seul ou en binôme

Évaluation : deux notes distinctes, une par matière

1. Principe

Chaque groupe développe une application complète en deux parties :

- une **API REST** qui expose un catalogue et gère les collections personnelles des utilisateurs authentifiés ;
- une **interface React** qui consomme cette API.

Le **contrat d'API est imposé** (section 5). Les deux matières peuvent donc avancer **en parallèle et indépendamment** : le frontend travaille sur des données de test conformes au contrat, le backend sur sa documentation `/docs`. Le branchement réel n'intervient qu'en dernière séance de chaque côté.

2. Choix de l'univers

Chaque groupe choisit **un** univers, déclaré en début de projet et identique des deux côtés :

Univers	Exemples de champs spécifiques
Bibliothèque de livres	auteur, année, nombre de pages, genre
Ludothèque de jeux vidéo	studio, plateforme, année, genre
Catalogue de recettes	temps de préparation, difficulté, type de plat

Le catalogue n'est **pas** récupéré sur une API externe : il est **alimenté par vos soins** via un script de peuplement (section 4.2).

3. Fonctionnalités attendues

3.1 Comptes

- Inscription (email + mot de passe), connexion, déconnexion.
- Un utilisateur ne voit **que sa propre collection**.

3.2 Catalogue (public)

- Recherche par mot-clé, filtre par catégorie, pagination.
- Fiche détaillée d'un élément.

3.3 Collection personnelle (authentifié)

Chaque entrée porte : un **statut** (`a_decouvrir` / `en_cours` / `termine`), une **note** de 1 à 5, un **commentaire** optionnel, une **date d'ajout** automatique.

Ajout, modification, suppression. Un même élément ne peut pas être ajouté deux fois.

3.4 Filtres et statistiques

- Filtre par statut, tri par date d'ajout ou par note.
- Page de statistiques : total, répartition par statut, note moyenne.

4. Partie Backend FastAPI

4.1 Compétences mobilisées

Le projet réutilise l'intégralité des modules travaillés en exercices :

Module travaillé	Où il sert dans le projet
Routing, path & query params	<code>GET /items</code> (recherche, filtre, pagination), <code>GET /items/{item_id}</code>
<code>Query()</code> et validation	bornes sur page, limit, longueur minimale de q
Pydantic, <code>Field</code> , <code>response_model</code>	tous les schémas d'entrée/sortie, modèle imbriqué <code>CollectionEntry.item</code>
<code>HTTPException</code> , codes de statut	201 à la création, 204 à la suppression, 404, 400, 409, 401
Handler d'exception maison	format d'erreur uniforme sur toute l'API
<code>Depends</code>	session de base, utilisateur courant, paramètres de pagination
Dépendance dans une dépendance	<code>get_current_user</code> dépend de <code>get_session</code>
<code>yield</code> (setup / teardown)	ouverture et fermeture de la session de base
JWT, hachage, expiration	inscription, connexion, protection des routes <code>/me/*</code>
<code>async / await</code> , ORM	toutes les routes et tous les accès à la base
Type hints (module d'onboarding)	obligatoires sur toute signature de fonction

4.2 Contraintes techniques

- Base **SQLite** en accès **asynchrone** (`SQLModel` + `aiosqlite`, ou `SQLAlchemy async`).
- Un script `seed.py` peuple le catalogue avec **au moins 40 éléments** de votre univers, répartis sur **au moins 4 catégories**. Relancer le script ne doit pas créer de doublons.
- Mots de passe hachés avec **bcrypt** (`passlib`). Aucun mot de passe, en clair ou haché, ne sort jamais de l'API.
- La clé de signature JWT et la configuration viennent d'un fichier `.env` **non versionné** (un `.env.example` est fourni à la place).
- **CORS** configuré pour autoriser l'origine du serveur de développement Vite (`http://localhost:5173`), et elle seule.
- Découpage en modules : `routers/`, `models/`, `schemas/`, `dependencies/`, `core/` (sécurité, config), `db/`.
- `main.py` ne contient **aucune route** : uniquement l'assemblage de l'application et l'inclusion des routers.
- La documentation `/docs` doit être exploitable : chaque route porte un `summary`, un `response_model` et ses codes de réponse déclarés.

5. Contrat d'API (imposé)

Préfixe : `http://localhost:8000`. Les routes `/me/*` exigent l'en-tête `Authorization: Bearer <token>`.

Authentification

Méthode	Route	Corps / Params	Réponse
POST	/auth/register	{email, password}	201 {id, email} — 409 si email déjà pris
POST	/auth/login	{email, password}	200 {access_token, token_type} — 401 si invalide
GET	/auth/me	—	200 {id, email} — 401

Catalogue (public)

Méthode	Route	Params	Réponse
GET	/items	q, categorie, page (≥ 1, défaut 1), limit (1–50, défaut 12)	200 {total, page, limit, results: Item[]}
GET	/items/{item_id}	—	200 Item — 404

Collection (authenti    )

Méthode	Route	Corps / Params	R��ponse
GET	/me/collection	statut, tri (date ou note)	200 Entry[]
POST	/me/collection	{item_id, statut, note?, commentaire?}	201 Entry — 404 item inexistant — 409 d���� pr��sent
PATCH	/me/collection/{entry_id}	{statut?, note?, commentaire?}	200 Entry — 404
DELETE	/me/collection/{entry_id}	—	204 — 404
GET	/me/stats	—	200 {total, par_statut: {...}, note_moyenne}

Sch  mas

```
// Item
{ "id": 1, "titre": "...", "categorie": "...", "description": "...",
  "image_url": "...", "annee": 1965 } // + 1 ou 2 champs propres    votre univers

// Entry — noter l'objet Item imbriqu  
{ "id": 7, "statut": "en_cours", "note": 4, "commentaire": "...",
  "date_ajout": "2026-03-14T10:22:00Z", "item": { /* Item */ } }
```

Format d'erreur (handler maison)

```
{ "erreur": { "code": 404, "message": "Item introuvable" } }
```

6. Partie Frontend React + TypeScript

6.1 Contraintes techniques

- `strict: true` dans le `tsconfig.json`. Le mot-cl   `any` est **interdit** (`unknown` suivi d'une v  rification est accept   et valoris  ).
- Les types du contrat d'API sont d  crits dans `src/types/api.ts` et **  crits    la main** — pas de g  n  ration automatique depuis l'OpenAPI.

- Statut typé en union littérale :

```
type Statut = "a_decouvrir" | "en_cours" | "termine";
```

- Composants fonctionnels uniquement, props typées explicitement, `key` stable jamais issue de l'index.
- Deux hooks personnalisés minimum**, dont un générique :

```
function useLocalStorage<T>(cle: string, valeurInitiale: T): [T, (v: T) => void]
```
- Deux Contexts** : `AuthContext` (token + utilisateur courant) et `CollectionContext`.
- React Router**, avec des **routes protégées** : `/collection` et `/stats` redirigent vers `/login` si aucun token valide.
- Tous les appels réseau passent par un **client HTTP unique** dans `src/services/` qui ajoute l'en-tête `Authorization` et traduit le format d'erreur de l'API en type TypeScript exploitable. Aucun `fetch` ailleurs dans le code.
- Les trois états **chargement / erreur / vide** sont traités sur chaque écran qui appelle l'API.
- Debounce* d'environ 400 ms sur le champ de recherche.
- Aucune librairie de composants prêts à l'emploi (MUI, Chakra, Bootstrap). CSS, CSS Modules ou Tailwind au choix.
- Lisible en largeur 375 px. Aucun composant au-delà de 150 lignes.

7. Note sécurité (transversale)

Ces points sont évalués des deux côtés :

- Mot de passe **jamais** stocké ni journalisé en clair ; bcrypt, pas de MD5 ni SHA-256 nu.
- Le hash de mot de passe ne figure dans **aucun** `response_model`.
- Secret JWT hors du dépôt Git. Un secret committé = la note de sécurité à zéro.
- Token expirant (durée courte, à justifier en soutenance).
- CORS restreint à l'origine de développement, jamais `allow_origins=["*"]` avec `allow_credentials=True`.
- Aucune requête SQL construite par concaténation de chaînes.
- Une entrée de collection n'est modifiable ou supprimable que par son propriétaire : **vérifiez-le côté serveur**, masquer le bouton dans l'interface ne protège rien.
- En soutenance, sachez expliquer le risque lié au stockage du JWT en `localStorage` et l'alternative envisageable.

8. Rendu

- Un seul dépôt Git**, deux dossiers : `api/` et `web/`, chacun avec son `README.md`.
- Historique de commits réparti sur les séances — un commit final unique est sanctionné.
- `node_modules/`, `.venv/`, `.env` et le fichier `.db` ne sont pas versionnés.
- Le `README` racine décrit la procédure de lancement complète : installation, peuplement, démarrage des deux serveurs.
- L'application doit démarrer sur une machine vierge en suivant uniquement ce README.
- Soutenance commune de 15 minutes : démonstration bout en bout, puis explication du code sur un point choisi par chaque intervenant.

9. Barème

Note Python / FastAPI (/20)

Critère	Points
Routes du contrat implémentées et conformes (codes de statut compris)	6
Modélisation Pydantic / SQLAlchemy, <code>response_model</code> , validation	4
Dépendances (<code>Depends</code> , imbrication, <code>yield</code>) et découpage en modules	3
Authentification JWT fonctionnelle et correctement protégée	3
Sécurité (section 7)	2
Type hints, lisibilité, README, <code>/docs</code> exploitable	2

Note React / TypeScript (/20)

Critère	Points
Fonctionnalités réalisées sans bug bloquant	6
Qualité du typage (aucun <code>any</code> , union littérale, hook générique, types du contrat fidèles)	4
Architecture (client HTTP unique, Contexts, hooks, routes protégées, découpage)	4
États chargement / erreur / vide traités partout	2
Interface soignée, cohérente, responsive	2
Git et README	2

Pénalités communes : contrat d'API non respecté d'un côté ou de l'autre (-3 pour le côté fautif) · projet ne démarrant pas en suivant le README (-4) · secret versionné (-3).

Bonus (+2 max, par matière) : tests (`pytest` / `vitest`) · rafraîchissement de token · thème clair / sombre · déploiement en ligne · défilement infini · conteneurisation Docker.